# Deploy Your Own Cryptocurrency

## Hands-On ERC-20 Lab & Deep Dive

Standalone Technical Lab

---

*"From zero to token in 15 minutes"*

Prof. Dr. Joerg Osterrieder

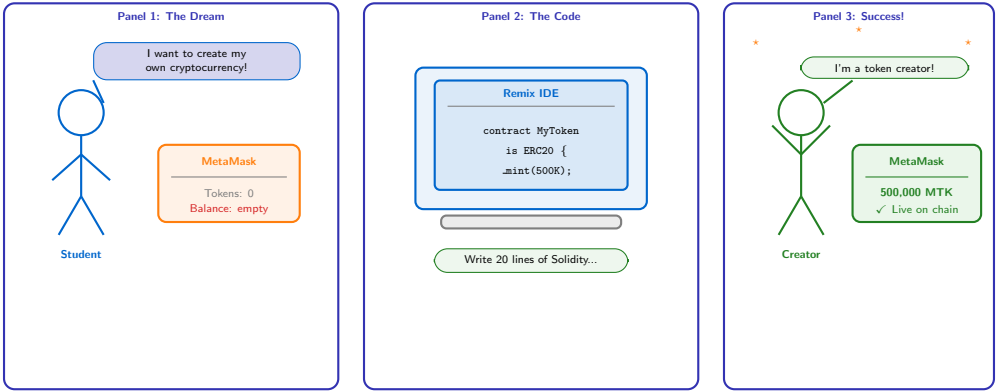
University Lecture Series

March 12, 2026

# Part 1: Deploy Your Own Cryptocurrency

A step-by-step hands-on lab using Remix IDE and Sepolia testnet

# The Mission




**Creating a token is writing a smart contract — anyone can do it in minutes.**

# Step 1 — The Complete Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

contract MyToken is ERC20, Ownable {
    uint256 public constant MAX_SUPPLY
        = 1_000_000 * 10**18;

    constructor()
        ERC20("MyToken", "MTK")
        Ownable(msg.sender)
    {
        _mint(msg.sender, 500_000 * 10**18);
    }

    function mint(address to, uint256 amount)
        public onlyOwner
    {
        require(
            totalSupply() + amount <= MAX_SUPPLY,
            "Exceeds cap"
        );
        _mint(to, amount);
    }

    function burn(uint256 amount) public {
        _burn(msg.sender, amount);
    }
}
```

**Name & Symbol**
"MyToken", "MTK"
Human-readable identity

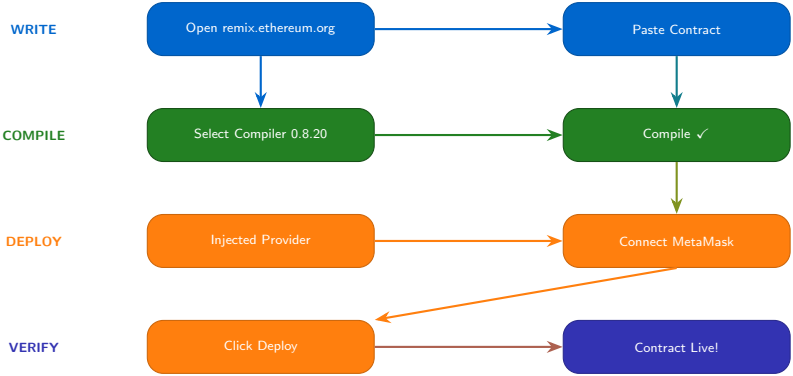
**Initial Supply**
500K of 1M cap (50%)
Minted to deployer

**Mint (Owner Only)**
Can mint remaining 500K
Enforces MAX_SUPPLY cap

**Burn (Anyone)**
Any holder can burn
their own tokens

**This contract gives you a capped, mintable, burnable ERC-20 token — 500K minted now, 500K mintable later.**

# Step 2 — Deploy on Remix




**Remix IDE handles compilation and deployment — no local tooling required.**

# Step 3 — Verify & Interact

**Panel 1: Contract is Live**

**Etherscan**

`0x7a3B...9F2d`

MyToken (MTK)

ERC-20 Token

✓ Verified

**Your contract is live!**

On-chain forever

**Panel 2: Read Contract**

**balanceOf(yourAddress)**

`Returns:`

500000000000000000000000

That's 500K tokens
with 18 decimals!

$500{,}000 \times 10^{18}$
= 500K human-readable

**Panel 3: Write Contract**

**transfer()**

`to: 0xFriend...`

`amount: 100*10^18`

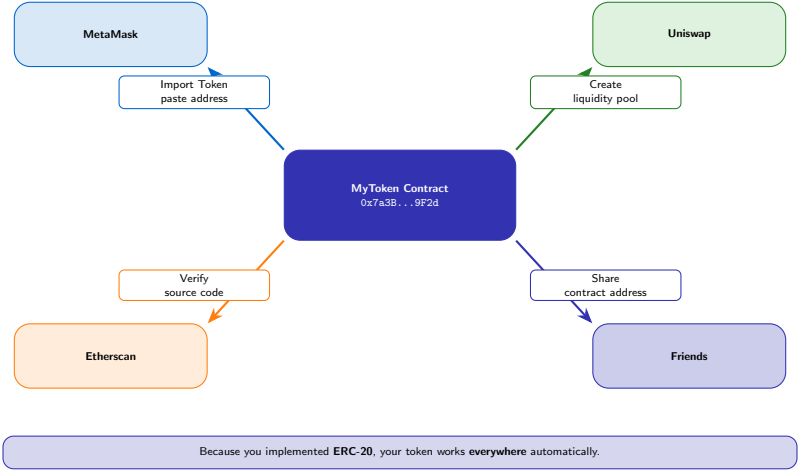
✓ **TX Success!**
**Sent 100 MTK!**

Gas used: ∼65,000

Confirmed in ∼12 sec

**Once deployed, anyone can read your contract and interact with your token.**

# Step 4 — Add to MetaMask & Share




**ERC-20 compliance means instant compatibility with the entire Ethereum ecosystem.**

# Part 2: Deep Dive — The Technical Details

Formal explanations behind five key concepts from the visual introduction

# How Tokens Actually Work — The Balance Mapping

**State: _balances**

| Address | Balance |
|---|---|
| 0xAlice | 500,000 |
| 0xBob | 10,000 |
| 0xCarol | 5,000 |

Total: 515,000 MTK

**transfer(Bob, 100)**

require(500K >= 100) ✓

Alice: 500,000

−100 = 499,900

Bob: 10,000

+100 = 10,100

**After Transfer**

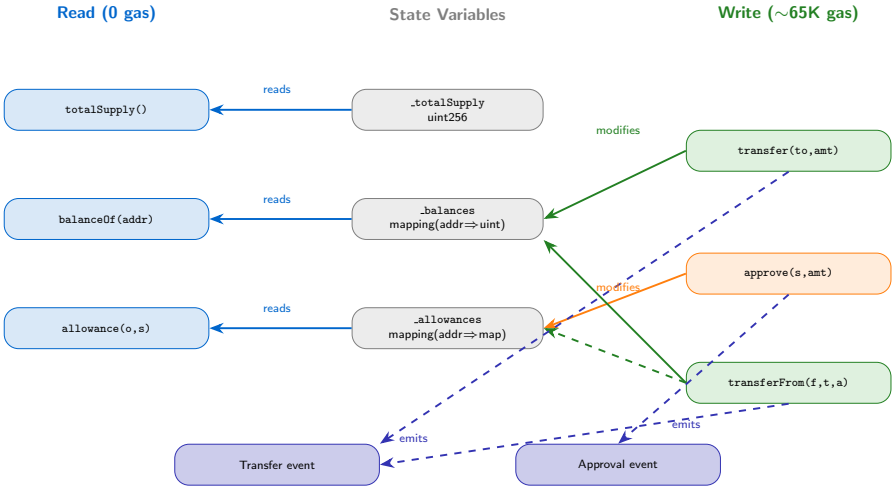
| Address | Balance |
|---|---|
| 0xAlice | 499,900 |
| 0xBob | 10,100 |
| 0xCarol | 5,000 |

Total: 515,000 MTK

```
mapping(address => uint256) private _balances;
function transfer(address to, uint256 amount) public returns (bool) {
    require(_balances[msg.sender] >= amount, "Insufficient");
    _balances[msg.sender] -= amount;
    _balances[to] += amount;
    emit Transfer(msg.sender, to, amount);
    return true;
}
```

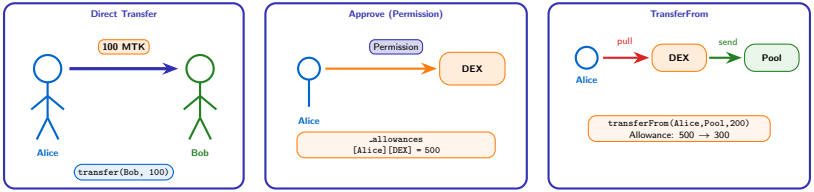
**A token is just a mapping from addresses to balances — the smart contract is the ledger.**

# The ERC-20 Interface — All 6 Functions




**Read functions cost zero gas; write functions require a transaction and pay gas.**

# Transfer vs Approve — The Full Mechanism




**Race condition warning: set allowance to 0 first, then set to new value**

```
mapping(address => mapping(address => uint256))
    private _allowances;
function approve(address spender, uint256 amount)
    public returns (bool) {
    _allowances[msg.sender][spender] = amount;
    emit Approval(msg.sender, spender, amount);
    return true;
}
```

**approve()+transferFrom() is the delegation pattern that powers all of DeFi.**

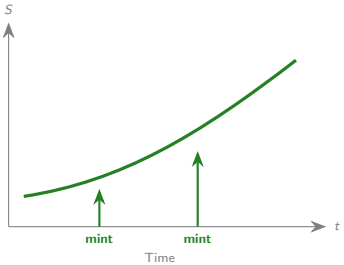
# Token Supply Models — The Mathematics

### Fixed Supply

$$S(t) = S_0, \quad \forall\, t \geq 0$$

```
uint256 public constant
  MAX_SUPPLY = 1_000_000 * 10**18;
```

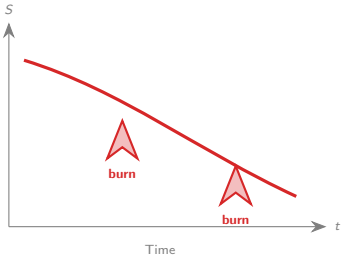
### Inflationary




$$S_n = S_0 + \sum_{i=1}^{n} m_i$$

```
function mint(address to, uint256 a)
  onlyOwner { _mint(to, a); }
```

### Deflationary



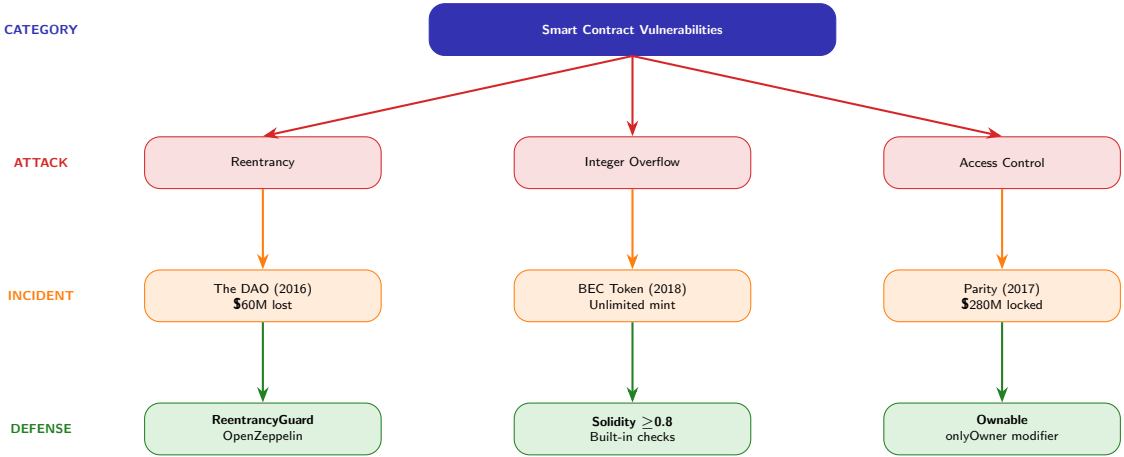

$$S_n = S_0 - \sum_{i=1}^{n} b_i$$

```
function burn(uint256 amt) public {
    _burn(msg.sender, amt); }
```

---

**Tokenomics = mathematical models governing supply evolution via mint and burn events.**

# Security — Why OpenZeppelin?




**Smart contract bugs are permanent and immutable — use audited libraries, always.**

# Key Terms for Beginners

| Term | Definition |
|---|---|
| **Blockchain** | A shared, tamper-proof digital ledger that records every transaction across a network of computers. |
| **Ethereum** | A blockchain platform that supports smart contracts and is home to most ERC-20 tokens. |
| **Smart Contract** | A self-executing program on the blockchain that enforces rules automatically, without intermediaries. |
| **Token** | A digital asset created by a smart contract on an existing blockchain (unlike a coin, which has its own chain). |
| **ERC-20** | The standard interface (6 functions + 2 events) that all fungible tokens on Ethereum must implement. |
| **Fungible** | Interchangeable; one token equals any other of the same type, like dollar bills. |
| **Solidity** | The programming language used to write smart contracts on Ethereum. |
| **Remix IDE** | A free, browser-based tool for writing, compiling, and deploying Solidity smart contracts. |
| **Gas** | A fee (paid in ETH) for executing operations on Ethereum; every transaction costs gas. |
| **Wallet** | Software (e.g., MetaMask) that stores your private keys and lets you interact with the blockchain. |

| Term | Definition |
|---|---|
| **Deploy** | Sending your compiled contract to the blockchain, creating it at a unique address forever. |
| **Testnet** | A practice blockchain (e.g., Sepolia) where you can deploy and test for free, without real money. |
| **MetaMask** | The most popular browser wallet for interacting with Ethereum and ERC-20 tokens. |
| **OpenZeppelin** | A library of audited, battle-tested smart contract code that prevents common security bugs. |
| **Mapping** | A Solidity data structure (like a dictionary) that stores key-value pairs, e.g., address $\longrightarrow$ balance. |
| **Mint** | Creating new tokens, increasing the total supply. Typically restricted to the contract owner. |
| **Burn** | Permanently destroying tokens, decreasing the total supply. |
| **Approve** | Granting another address (e.g., a DEX) permission to spend your tokens on your behalf. |
| **Tokenomics** | The economic design of a token: supply model, distribution, incentives, and value. |
| **Etherscan** | A block explorer website where you can view transactions, contracts, and token balances. |

**Master these 20 terms and you have the vocabulary to understand any ERC-20 project.**

# Key Takeaways

## Deploy Your Own Cryptocurrency: Summary

1. **Deploy in 15 minutes:** OpenZeppelin + Remix + Sepolia = your own token live on-chain.
2. **The balance mapping is the heart:** `mapping(address => uint256)` is the entire ledger.
3. **6 functions + 2 events = universal compatibility** with every wallet and DEX.
4. **approve()+transferFrom() enables DeFi composability** — the delegation pattern.
5. **Supply model math determines token economics** — choose fixed, inflationary, or deflationary.

**Next: Token economics deep dive and real-world deployment strategies.**

---

**You now have the knowledge to create, deploy, and explain your own ERC-20 token.**